



ADR-001: Hexagonal (ports & adapters) / Clean architecture

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

- **Status:** Accepted
- **Date:** 2026-10-07
- **Deciders:** iorta TechNXT solution architect, TASCO IT architecture
- **Related:** [ADR-002](#), [ADR-004](#), [ADR-005](#)

Context and problem statement

On day one, most of the systems this platform must call are not available to integrate against:

- the VETC wallet and VETC data feed;
- TASCO core policy administration;
- Zalo ZNS, the SMS brandname provider and app push;
- a Vietnamese voice-AI vendor;
- the TASCO IdP.

Even so, the business logic has to be demonstrable, testable and auditable from the start. That logic covers golden-record build, scoring, journeys, contact policy, rating and the dialogue policy. When each real system arrives, swapping it in must not touch that logic.

Decision drivers

- The pilot must run end-to-end with sandbox integrations, and production must swap adapters without regression.
- Domain logic must be deterministic and unit-testable without I/O. Today's date is injected (`src/shared/clock.js`).
- Regulated logic needs one obvious place to review: tariff, copy guard, consent.
- The team is small, so the structure has to be simple enough to keep consistent.

Considered options

1. **Hexagonal / Clean architecture:** domain → application → ports → adapters, with a single composition root.
2. Classic layered MVC (controllers → models → DB).
3. Microservices per capability (ingestion, leads, journeys, sales, voice).

Decision outcome

Chosen option: **1. Hexagonal**, implemented as follows.

Ring	Path	Allowed dependencies
Shared kernel	<code>src/shared/*</code>	<code>node:*</code> only
Domain + rules	<code>src/domain/*</code> , <code>src/rules/*</code>	shared kernel
Application (use cases)	<code>src/application/*</code>	domain, rules, shared. Ports arrive as factory arguments.
Adapters	<code>src/adapters/{http,persistence,messaging,integrations}</code>	application, domain, shared

Ring	Path	Allowed dependencies
Composition root	<code>src/bootstrap/container.js</code>	everything. This is the only place that picks adapters.

Ports are plain JavaScript object contracts documented with JSDoc and in [integration-architecture.md](#). Each outbound integration is wrapped in a circuit breaker at composition time (`breaker(name, port, opts)` in `container.js`).

Consequences

- Good: a sandbox adapter can be replaced by a real one in a single place, for example `createVetWalletGateway` → a real wallet client.
- Good: the in-memory store makes tests and demos fast, and the Postgres adapter gives the same behaviour ([ADR-004](#)).
- Good: the domain modules are pure, so the scoring, MDM and dialogue rules can be reviewed in isolation by compliance and model risk.
- Bad: ports are implicit (duck-typed JS) and there is no compile-time check. Contract tests per adapter are required (planned: a `test/contract/*` suite run against both stores and each real adapter in SIT).
- Bad: some HTTP handlers in `routes.js` still hold small amounts of orchestration (partner onboarding, steward expiry correction, voice campaign loop). **Follow-up:** move these into application services so the rule "HTTP adapter only translates" holds strictly.
- Neutral: one deployable (a modular monolith). Each application service has its own collection set, so it can be split later along those service boundaries if scale or team topology demands it.

Pros and cons of the options

Layered MVC. Familiar. However, persistence and HTTP concerns leak into business logic, and sandbox-vs-production switching would be scattered across the code.

Microservices. Independent scaling and deployment. However, the team and the pilot do not justify distributed transactions, a service mesh, or several pipelines and databases yet. Hexagonal keeps that option open.